CS1200: Intro. to Algorithms and their Limitations Profs. Anshu and Nesson

Problem Set 5

Harvard SEAS - Fall 2026 *Due: Mon Oct. 26, 2026 (11:59pm)*

Please see the syllabus for the full collaboration and generative AI policy, as well as information on grading, late days, and revisions.

All sources of ideas, including (but not restricted to) any collaborators, AI tools, people outside of the course, websites, ARC tutors, and textbooks other than Hesterberg–Vadhan must be listed on your submitted homework along with a brief description of how they influenced your work. You need not cite core course resources, which are lectures, the Hesterberg–Vadhan textbook, sections, SREs, problem sets and solutions sets from earlier in the semester. If you use any concepts, terminology, or problem-solving approaches not covered in the course material by that point in the semester, you must describe the source of that idea. If you credit an AI tool for a particular idea, then you should also provide a primary source that corroborates it. Github Copilot and similar tools should be turned off when working on programming assignments. If you did not have any collaborators or external resources, please write 'none.'

Your name: Andrew Jing
Collaborators and External Resources: Claude for formatting solution sections
No. of late days used on previous psets: 1
No. of late days used after including this pset: 1

1. (Shortest Path without predecessor array)

   Home Depot is providing a navigation service for delivery drivers who all leave from the same depot s. The road network is a directed, unweighted graph $G = (V, E)$, with n vertices and m edges, given by outgoing adjacency lists. Every road takes one unit of time to traverse. The graph and depot remain fixed throughout the day. Each query specifies a destination t and asks for a shortest path from s to t, or $\perp$ if none exists.

   Unfortunately, the server that holds the predecessor array has crashed. The backup drive still has the graph, the distance array, and a log of every frontier in the BFS starting from s:

   $$F_i = \{v : \text{dist}_G(s, v) = i\}.$$

   Each frontier is saved as a list of vertex names. You may assume that accessing the start of any frontier list takes constant time in the Word-RAM model.

   The engineers deploy the following emergency query algorithm that runs on the back-up drive data:

   - If $\mathsf{dist}[t] = \infty$, return $\perp$.
   - Otherwise, set $d = \mathsf{dist}[t]$, start a 'path' array containing t, and set $v = t$.
   - For $i = d, d-1, \ldots, 1$, scan the vertices u in F_{i-1} and their outgoing adjacency lists until finding an edge (u, v). Append u to the path list and set $v = u$.
   - After the path array appends s, reverse the path array and return it.

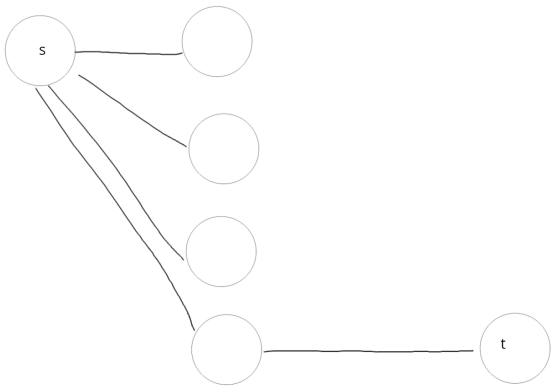
Answer these questions:

(a) Prove that the emergency algorithm returns a shortest path whenever one exists.

Solution. For the case where $dist[t] = \infty$, by definition there is no path from s to t, meaning the $\perp$ result is correct.
For finite distances, we know there exists a path of length $dist[t]$. For each iteration in the loop, $dist[v] = i$, so $v \in F_i$, and since F_i was constructed from F_{i-1} we know there exists a u with $dist[u] = i - 1$ and an edge (u, v). That means that every iteration of the loop does indeed append exactly one vertex to the path array, and that each new element has a distance 1 less than the last. Also, we know that the last appended element is definitely s – if $d = 0$ then the loop is skipped and the path array only has t (which equals s in this case), and otherwise the last item is appended from F_0, which only contains s. Reversing the array doesn't change it's length, and the algorithm returns an array with $d+1$ elements, one vertex for each loop iteration plus t. After reversing, each consecutive pair in the array has an edge going from left to right, and the array starts at s and ends with t. With $d + 1$ elements, we know there is a length d path from start to finish, and by definition this is a shortest path because $d = dist[t]$.

(b) Analyze the time to find the shortest path in the emergency algorithm, in terms of n, m. Can a finding a shortest path take $\Omega(n)$ time even when the returned path has only two edges? Justify your answer with a graph. Argue for or against the value of predecessor array (which is not accessible to the emergency algorithm above) in finding a shortest path.

Solution. Let's assume finding $dist[t]$. This means the first two steps in the algorithm are $O(1)$. The d loop iterations each check $O(|F_{i-1}|)$ vertices for an edge with v with $O(deg(u))$ time, along with a few constant-time things. As each vertex occurs in one frontier set and each edge occurs in one vertex's adjacency list, if x is the number of vertices with distance $< dist[t]$, then the loop takes $O(x + \Sigma deg(x))$. In terms of n and m this is $O(n + m)$.



This graph is an example of how a distance 2 vertex can still need to go through all the vertices in F_1 to find its predecessor, and since s and t also need to be visited, we've

gone through $O(n)$ operations for this short path.

Predecessor arrays are a clear win for an application like this with one source vertex, dropping the needed runtime from $O(n+m)$ to $O(d)$ with an extra $O(n)$ space cost. We can simply traverse the predecessor chain backwards to make the array without exhaustively looking through the entire previous frontier. Unless space is a bottleneck, then this is beneficial.

Please keep your response within 1 page.

2. (Solving Games) Consider playing a solo game on an $n \times n$ chessboard. You have one piece, a chess knight, which starts in the lower-left corner, and your goal is to reach any of the other three corners in as few moves as possible. Like a usual chess knight, in one move, you can move to any position that is two squares away in a horizontal direction and one square away in a vertical direction, or two squares away in a vertical direction and one square away in a horizontal direction. There is a catch, however: some squares have visible landmines, so you cannot move to them (since you do not want set off an explosion).

   (a) Give an algorithm that achieves the above goal in time $O(n^2)$ by reduction to either the ShortestWalks problem or the SingleSourceShortestPaths problem. The algorithm should output $\perp$ if no sequence of moves can take you to any of the other three corners when started in the lower-left corner. Discuss correctness and runtime of the algorithm. *Please keep your response within 1 page.*(Note: if reducing to SingleSourceShortestPaths, we haven't defined abstractly what it means to reduce a computational problem to a data structure problem, but you may construct and use a SingleSourceShortestPaths data structure on an appropriately constructed graph.)

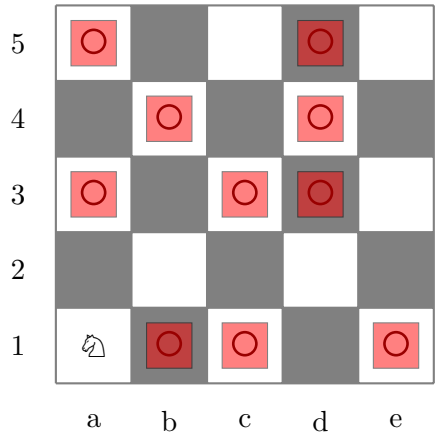
   Solution.

   1. Construct a graph by defining n^2 vertices in the set V, one for each square.
   2. For each non-landmine vertex
   3. Add an edge to the set E for every valid move of the knight from that square (not to a landmine square).
   4. For all 3 target corners
   5. $p =$ SingleSourceShortestPaths$((V, E), start, target)$
   6. If $p \neq \perp$ return p
   7. return $\perp$

   Since each square has at most 8 valid moves from it, we have $|E| = O(n^2)$. Then, we reduce to SingleSourceShortestPaths, returning $\perp$ if all three other corners' shortest paths are $\perp$ from the Oracle and the first returned path otherwise.

   This algorithm is correct because the knight's position is equivalent to a state machine (graph) representing current position, and the only way to change states is to execute a valid move. Then, calculating a shortest path to a target state is solved by reducing to SingleSourceShortestPaths on (at most) all of the target states. The cost is that this reduction requires constructing the state machine (graph) by constructing n^2 vertices and $O(n^2)$ edges for a total $O(n^2)$ runtime and at most 3 Oracle calls (the $O(n)$ copies during the p assignment are dominated by graph construction).

(b) Carry out your algorithm on the 5×5 board shown below, listing both the frontier vertices and the predecessor relationships at each stage of BFS (the red square symbols are landmines).



Solution.

1. Squares a1 through e5 get their own vertices
2. Valid edges are formed (e.g. (a1,b3) but not (b2, d3))
3.1. Calling the Oracle on top-left and bottom-right corners returns $\perp$.
3.2. Calling the Oracle on top-right triggers BFS.
3.2.1 F_0 ={a1} and pred[] is empty
3.2.2 F_1 ={b3,c2}, pred[b3]=pred[c2]=a1.
3.2.3 F_2 ={c5,d2,e3}, pred[c5]=pred[d2]=b3, pred[e3]=c2
3.2.4 F_3 ={a4,e4,c4,d1}, pred[a4]=pred[e4]=[c5], pred[c4]=d2, pred[d1]=e3
3.2.5 F_4 ={b2,e5}, pred[b2]=a4, pred[e5]=c4
3.2.6 p = (e5,pred[e5])
3.2.7 p = (e5,c4,pred[c4])
3.2.8 p = (e5,c4,d2,pred[d2])
3.2.9 p = (e5,c4,d2,b3,pred[b3])
3.2.10 p = (e5,c4,d2,b3,a1)
3.2.11 p = (a1,b3,d2,c4,e5)

3. (Maximal Independent Sets) Let $G = (V, E)$ be a graph. A set $S \subseteq V$ is a *maximal* independent set if we cannot add any vertices to S while it remains an independent set. That is, for every vertex $v \in V \setminus S$, $S \cup \{v\}$ is not an independent set.

   Given a graph G, a maximal independent set S can be found in time $O(n + m)$ greedily. Specifically, we can start with an arbitrary node in V and add it to S; iterate through each of that node's neighbors and remove them from V; and repeat with the next remaining element in the vertex set until all nodes are exhausted (you can take this fact as a given). Note that this is in sharp contrast to *maximum-size* independent sets, for which we do not know any subexponential-time algorithms.

We also know that if G is 3-colorable, then it has a 3-coloring f in which the set of vertices of color 2 (i.e. $f^{-1}(2)$) is a maximal independent set (you can also take this fact as a given). Further fact is that every graph G has at most $3^{n/3}$ maximal independent sets, and there is an algorithm (the Bron–Kerbosch algorithm) that enumerates all of the maximal independent sets in time $O(3^{n/3})$. Use these facts to show that a 3-coloring can be solved in time $O((n+m)\cdot 3^{n/3}) = O(1.45^n)$, improving the runtime of $O(1.89^n)$ from Lecture 12. *Please keep your response within 10 sentences.*

Solution.

1. I = Bron-Kerbosch algorithm on G ($|I| \leq 3^{n/3}$)
2. For $S \in I$
3. $\quad T = \text{BFSColor}(G \setminus S)$
4. $\quad$ If $T \neq \perp$ return $T \cup S$, where T is colors 0 and 1 and S is color 2
5. return $\perp$

Here we use a version of BFSColor that returns $\perp$ the instant it assigns the color 3 (so it doubles as a 2-colorability detector). Since G is 3-colorable $\implies$ $f^{-1}(2)$ is maximally independent, in other words G is 3-colorable $\implies$ $\exists$ maximally independent S with color 2 s.t. $G \setminus S$ is 2-colorable. So, we can check all maximally independent sets' complements for 2-colorablility (i.e. attempt a 3-coloring of G). If we exhaust all maximally independent sets without finding one, then the contrapositive of the earlier statement implies there is no 3-coloring of G. For each of the $O(3^{n/3})$ maximally independent subsets, generating S and $G \setminus S$ takes a total of $O(n)$ time and checking for 2-colorability in $G \setminus S$ takes $O(n+m)$. So in total the loop takes $O((n+m)\cdot 3^{n/3})$ time, and other factors like return construction are absorbed into that runtime.

4. (Reflection) Take one homework problem you have worked on this semester that you struggled to understand and solve, and explain how the struggle itself was valuable. In the context of this question, describe the struggle and how you overcame the struggle. You might also discuss whether struggling built aspects of character in you (e.g. endurance, self-confidence, competence to solve new problems) and how these virtues might benefit you in later ventures.

   Solution. The first two psets of the semester were particularly helpful in establishing the tone of the proofs expected in CS 1200. Specifically, I remember struggling with the level of detail in my proofs, since coming CS 20 (where it's best to cite all lemmas, establish all nitty-gritty caveats, etc.) it was difficult to transition to the size of the proofs needed in this class. So the struggle at the beginning of this semester helped me become more adaptable to the needs of this particular class. That means I'm able to work a little bit smarter rather than blindly sticking to a preconcieved self-expectation for this class's homework.

5. Once you're done with this problem set, please fill out this survey so that we can gather students' thoughts on the problem set, and the class in general. It's not required, but we really appreciate all responses! **This week would be a great time to provide any mid-semester feedback, which could help us improve the course in the second half of the semester.**